

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ Информатики и систем управления

КАФЕДРА Теоретической информатики и компьютерных технологий

ОТЧЕТ ПО ПРОИЗВОДСТВЕННОЙ ПРАКТИКЕ

Студент Дьячков Егор Сергеевич

Группа ИУ9-61Б

Тип практики Производственная

Название предприятия ПАО "Сбербанк России"

Студент _____ Дьячков Е.С.
подпись, дата *фамилия, и.о.*

Рекомендуемая оценка: _____
Руководитель практики
от предприятия: _____ Жиров Д.В.
подпись, дата *фамилия, и.о.*

Руководитель практики _____ Коновалов А.В.
подпись, дата *фамилия, и.о.*

Оценка _____

2025 г.

Содержание

Содержание 2
Введение 3
Характеристика предприятия 4
Отчет о проделанной работе 6
Разработка сайта для визуализации графа и загрузки книг 6
Разработка сервера, объединяющего сервисы для анализа книг 9
Заключение 12

Введение

Производственная практика является важнейшим этапом в подготовке специалистов, так как она позволяет закрепить теоретические знания, полученные в процессе обучения, и применить их в реальных условиях профессиональной деятельности. Практика дает возможность студентам познакомиться с современными технологиями, процессами организации работы получить опыт решения реальных задач и улучшить навык работы в команде.

Данный отчет отражает результаты прохождения производственной практики, организованной на базе публичного акционерного общества «Сбербанк России». В ходе практики внимание было сосредоточено на задаче выделения терминов, относящихся к области кибербезопасности, из различной профессиональной литературы для последующего добавления их в граф знаний. Данный граф имеет высокую значимость, так как благодаря нему можно отслеживать, какие области кибербезопасности уже известны, а какие еще нет. Если какая-то область является новой в графе, то ее можно легко визуально увидеть на графе знаний, так как она будет помечена другим цветом.

Основной целью производственной практики стало исследование и разработка подходов к распознаванию и выделению терминов, которые относятся к области кибербезопасности, среди других различных других терминов или слов, а также построение и визуализация графа знаний. Для выполнения данной цели поставленная задача была разделена на несколько подзадач, решение которых позволило постепенно создать сервис с полным циклом обработки входящего файла и последующей визуализацией графа знаний на сайте.

В дальнейшем отчете подробно изложен ход выполнения поставленных заданий, приведено описание разработанного программного кода и представлены результаты проведенной работы на каждом этапе. Полученное решение может быть использовано, как инструмент, для выявления новых областей кибербезопасности из профессиональной литературы.

Характеристика предприятия

Сбер — исторический преемник сберегательных касс Российской империи, а затем и советских Гострудсберкасс. 12 ноября 1841 года император Николай I подписал указ об учреждении в России сберегательных касс «для доставления через то средств к сбережению верным и выгодным способом». Первые кассы открываются в 1842 году в Санкт-Петербурге, Москве и Одессе. На сберегательных книжках печатается изображение кормящего птенцов пеликана как символ заботы о наименее обеспеченных людях.

Первым клиентом первой российской сберегательной кассы стал служащий Ссудной казны в Санкт-Петербурге Николай Кристофари. 1 марта 1842 года он открыл вклад на 10 рублей — максимально возможную в то время в сберкассах сумму. В Санкт-Петербурге и Москве ему установлены памятники, выражающие уважение Сбера к своим клиентам. К концу 1850-х годов вкладчиками сберкасс стали 90 тысяч человек.

Император Александр II утвердил 16 октября 1862 г. новый Устав сберегательных касс, по которому они должны были создаваться «во всех городах, посадах и местечках». В результате в России появилась сеть финансовых учреждений, доступных для всех сословий. К 1895 году было открыто 3875 касс, на вкладах хранилось 368 млн рублей. Это время называют эпохой расцвета сберегательного дела в России.

В 1895 году устав сберегательных касс снова изменён: правительство принимает на себя полную ответственность за сохранность вверенных кассам вкладов. В 1905 году кассы получили право выдавать страховые полисы. В 1902 году сберегательные кассы открываются практически повсюду. Были они и на кораблях военного флота, в том числе на знаменитых крейсерах «Варяг» и «Аврора».

После Октябрьской революции в 1919 году сберегательные кассы объединили с Народным банком РСФСР, а в 1925 году всю сеть преобразовали в Гострудсберкассы СССР. В период с 1922 по 1941 год в СССР были выпущены

различные облигации государственных займов, которые активно размещались через сберкассы, а полученные средства направлялись на индустриализацию советского государства.

В годы Великой Отечественной войны сберкассы сыграли важную роль в распространении военных займов и приёме пожертвований в Фонд обороны. Четыре военных займа под лозунгом «Всё для фронта, всё для победы!» собрали 72 млрд рублей. За счёт поступлений по займам покрыто 15% военных расходов. В послевоенное время сберкассы массово распространяют займы восстановления и развития народного хозяйства.

За тридцать лет количество сберегательных касс увеличилось вдвое: с 40 тысяч в 1951 г. до 79 тысяч в 1981. К началу 1986 года в них на 171 млн счетов хранились сбережения на сумму свыше 220 млрд рублей. После того как в 1963 году в сберкассах начали принимать коммунальные, налоговые, страховые и другие платежи, выдавать зарплату рабочим и служащим предприятий, их клиентами становятся практически все граждане СССР.

В 1987 году в ходе перестроечных реформ сберегательные кассы были реорганизованы в Сберегательный банк СССР. Начинается эмиссия банковских карт, появляются новые виды вкладов: срочные, молодёжные, на детей, накопительные. Сбербанк стал совершать обмен валюты, а в 1988 году открыл в Москве первый банкомат, ставший отправной точкой для самой большой в России сети устройств самообслуживания.

После ряда реорганизаций в 1991 году был зарегистрирован Сбербанк России. В 1996 году собрание акционеров приняло концепцию развития до 2000 года, нацеленную на его трансформацию в универсальный коммерческий банк. Сбербанк — ведущий участник рынка государственных ценных бумаг, крупный кредитор на рынке ссудных капиталов. Он выходит на международную арену, приобретая дочерний банк в Казахстане в 2006 году.

В ноябре 2007 года Президентом, Председателем Правления Сбербанка утверждён Герман Греф. Уже в следующем году Наблюдательный совет одобрил

концептуально новую стратегию развития, которая радикально изменила облик крупнейшего банка страны и предопределила его дальнейшую трансформацию в высокотехнологичную структуру.

В условиях пандемии Сбер определяет главную задачу: максимально помочь клиентам и государству. Он становится масштабным проводником государственных программ поддержки населения и бизнеса, предлагая и собственные решения. Сервисы телемедицины Сбера внедряются по всей стране, его ИТ-платформа используется для предоставления государственных услуг и создания высокотехнологичных экономик субъектов Российской Федерации.

Отчет о проделанной работе

Разработка сайта для визуализации графа и загрузки книг

Для выполнения были поставлены следующие задачи:

1. Изучить библиотеки для визуализации графов на React
2. Разработать дизайн сайта, опираясь на брендбук компании
3. Определить способы взаимодействия с сервером
4. Продумать архитектуру сервиса
5. Реализовать веб-сайт на React

В ходе практики был создан фронтенд приложения на основе фреймворка React, обеспечивающий удобный интерфейс для загрузки текстовых файлов и визуализации графа знаний. Выбор React был обоснован его значимостью на рынке фронтенд-разработки, потому что он остается наиболее популярным фреймворком благодаря зрелой экосистеме, большому сообществу, а также ключевым преимуществам таким, как компонентная архитектура, которая позволяет создавать переиспользуемые модули; оптимизация производительности и плавное обновление интерфейса без полной перезагрузки

страницы; высокая масштабируемость для крупных приложений; кросс-платформенная совместимость. Также рассматривались альтернативы такие, как Svelte или Vue, но React был выбран за его стабильность, богатую экосистему библиотек и способность справляться с сложными интерактивными UI, включая реальное обновление в реальном времени через WebSocket.

Для выбора подходящей библиотеки визуализации графов в React были проанализированы решения с учетом следующих требований: поддержка больших объемов данных, интерактивность, то есть возможность зума, перемещение узлов и выделение новых терминов цветом, производительность и простота интеграции. Рассмотрена vis-network - простая библиотека с поддержкой иерархических расположений графов, физики, кластеризации и событий, обеспечивающая легкость использования и производительность. Она была выбрана за готовые опции для ориентированных графов и автоматическую кластеризацию, идеально подходящую для графа знаний. Cytoscape - мощное решение для сложных графов, поддерживает кастомизацию и большие графы, но не была выбрана из-за высокой сложности. React-force-graph - легковесная библиотека на D3 с физикой и поддержкой 3D, она подходит для спиральных layouts, но уступает библиотеке vis-network в простоте из-за необходимости ручной настройки кластеров. Visx - гибкая библиотека для SVG/Canvas с низкоуровневым контролем. Эта библиотека использовалась для прототипов, но была заменена из-за необходимости глубоких знаний D3. Другие библиотеки, такие как Sigma, Graphin и ECharts, являются хорошими в производительности, но vis-network была выбрана за баланс простоты и функциональности. Ее зрелость, активное сообщество и прямая интеграция с данными, приходящими с сервера сделали ее оптимальным выбором для проекта.

Дизайн сайта был разработан с учетом брендбука Сбербанка, который подчеркивает современность и доступность через фирменные элементы: основной цвет - синий, для акцентов и границ узлов, дополнительный зеленый цвет нужен для выделения новых терминов, нейтральные тона для фона и для существующих узлов. В качестве шрифта был выбран современный и читаемый

Inter. Логотип Сбера размещен в хедере и футере, иконки upload, profile, social взяты из ассетов, соответствующих стилю, они минималистичные, в формате SVG. Сам интерфейс построен на принципах минимализма: карточки с тенями, закругленные углы и адаптивная сетка. Заголовки "Граф знаний", "Анализ текстового файла" сделаны крупными для лучшей читаемости. Прогресс-бар и кнопки используют hover-эффекты для интерактивности. Футер включает ссылки и иконки социальных сетей Telegram и VK. Отображение сайта с созданным дизайном было протестировано на десктопе и на мобильных устройствах.

Для взаимодействия с сервером были выбраны следующие протоколы: WebSocket и HTTP. Протокол WebSocket нужен для обработки загрузки файлов и получения данных от сервера в реальном времени. Запрос инициируется отправкой запроса "ws://62.109.7.163:8100/ws/upload" на сервер. Благодаря использованию данного протокола можно отправлять содержимое файла в формате .txt с выбранной моделью, BERT или LightGBM, получать промежуточные обновления такие, как прогресс и результаты анализа терминов, промежуточные итоги и финальный отчет. Низкая задержка, двусторонняя связь, обработка ошибок являются преимуществами и обусловили выбор протокола WebSocket. Для загрузки данных графа знаний в формате JSON был выбран протокол HTTP. Он используется для инициализации и обновлений визуализации по событию «graphUpdated». Также на сервере был настроен CORS, который позволяет добавлять необходимые заголовки, содержащие важную информацию об отправляемых данных, в header для того, чтобы фронтенд мог правильно обработать ответ.

Была выбрана модульная архитектура сервиса с разделением на слои: компоненты интерфейса, state management, data layer. Главный файл App.jsx — контейнер для роутинга. Layout.jsx — обертка для всех других файлов, содержащая хедер и футер. UploadSection.jsx — компонент для загрузки файла, интегрирующий WebSocket. GraphVisualization.jsx — рендеринг с ref для canvas, алгоритмом DFS для обработки различных классов терминов. Архитектура SPA,

без внешнего роутера и с кнопками для навигации, обеспечивает быструю навигацию.

Реализация включает в себя настройку проекта, Vite для быстрой сборки проекта, импорт ассетов и стилей. Компоненты собраны в директории "src/components": AboutPage.jsx — статическая страница с описанием; UploadSection — динамическая с FileReader и WebSocket; GraphVisualization — с vis-network, спиральным layout и кнопками увеличения/уменьшения масштаба. Main.jsx — точка входа. Функциональность: выбор файла/модели, обработка, рендеринг промежуточных и итоговых результатов, обновление графа по событию. Результатом работы над фронтенд частью является рабочий сайт, развернутый на сервере, с поддержкой загрузки файлов в формате .txt и визуализацией графа знаний с выделением зеленым цветом новых добавленных терминов. В итоге, можно сказать, что поставленные задачи по разработке сайта для визуализации графа и загрузки книг были выполнены в полном объеме.

Разработка сервера, объединяющего сервисы для анализа книг

Для выполнения были поставлены следующие задачи:

1. Выбрать язык программирования для написания сервера
2. Продумать архитектуру сервера
3. Продумать формат взаимодействия с внешними API
4. Выбрать фреймворк и реализовать хендлеры
5. Использовать технологию WebSocket для непрерывного обмена данными
6. Протестировать работу вместе с фронтендом
7. Контейниризовать каждый сервис и добавить их в общий docker-compose

В ходе практики была разработана серверная часть приложения на языке Go. Созданный сервер выступает в роли оркестратора, который объединяет внешние сервисы для анализа текстовых файлов и выделения из них терминов, проверки терминов и обновления графа знаний. Сервер обеспечивает полный

цикл обработки: от приема данных через WebSocket до агрегации результатов из специализированных внешних API и возврата промежуточных и итоговых данных клиенту. Выбор языка Go был обусловлен его высокой производительностью, возможностью выполнять параллельные запросы, статической типизацией для надежности и минималистичным синтаксисом, что значительно ускоряет разработку и отладку. Язык Go остается одним из лидеров в бэкенд-разработке для API и микросервисов, особенно в высоконагруженных системах. Его стандартная библиотека позволяет разработать веб сайт или API, не используя сторонние библиотеки. Альтернативы такие, как Node.js или Python рассматривались, но язык Go был выбран за скорость, низкое потребление памяти и пригодность для выполнения параллельных запросов в реальном времени.

Архитектура сервера построена по принципам чистой архитектуры с разделением на слои: handlers для обработки запросов, middlewares для логирования и CORS, logger для структурированного логирования и configs для загрузки настроек. Точка входа в сервис — файл main.go, где инициализируется конфигурация и сам сервер. Для маршрутизации сервер использует Chi роутер с graceful shutdown по сигналам SIGTERM/Interrupt. Благодаря использованию принципа graceful shutdown сервер останавливается постепенно, дожидаясь выполнения всех запросов, которые пришли до сигнала остановки, а также освобождает необходимые ресурсы. Данный подход реализуется с помощью применения контекста. Горутины при выполнении своей работы слушают канал, в который может передаться сигнал об отмене контекста, и в случае отмены контекста завершают свою работу, иначе продолжают свою работу до выполнения всех задач.

Формат взаимодействия определен как RESTful HTTP с JSON-payload, с использованием стандартного net/http клиента Go для запросов. Сервер выступает оркестратором и взаимодействует с несколькими внешними API. Для повышения производительности сервера все запросы используют таймаут 5 секунд, а также API-ключ для аутентификации и JSON-маршалинг. Для

общения с внешними API определены строгие форматы взаимодействия. Они представлены в виде структур с JSON тегами. Также строго определены форматы запроса и ответа для каждого хендлера на сервере. Это соответствует OpenAPI 3.0 и позволяет с помощью аннотаций перед хендлерами написать документацию, которую затем клиент может изучить. Альтернативные методы такие, как gRPC были отвергнуты, так как внешние API их не поддерживают. Для обеспечения безопасности производится валидация входных JSON запросов. Также добавлен CORS, добавляющий необходимые для фронтенда заголовки в ответ.

В качестве роутера был выбран легковесный роутер Chi, заменяющий стандартный mux для middleware и path params. Gorilla WebSocket для WS-upgrade. Swaggo для Swagger-документации. Использование технологии WebSockets позволяет непрерывного обмениваться данными с клиентом до закрытия соединения. Взаимодействие с помощью протокола WebSocket реализовано в хендлере UploadWebSocket. В начале клиент отправляет файл по сокету, сервер его принимает и отправляет по веб-сокетам прогресс обработки полученного файла в реальном времени. Это позволяет фронтенду обновлять компоненты без дополнительных запросов к серверу. Это является хорошим решением для длинных и долгих задач, которые могут выполняться несколько минут, так как клиенту важно отслеживать прогресс выполнения долгих задач. В итоге, был разработан надежный сервер на языке Go, объединяющий работу с внешними API. Он обеспечивает обработку данных в реальном времени, агрегацию данных и документацию с помощью Swagger, доступную для просмотра клиентам. Созданное решение масштабируемо и готово к доработкам. Например, можно добавить аутентификацию с помощью JWT токенов и мониторинг на Prometheus.

Для обеспечения удобства развертывания и масштабирования разработанных сервисов была реализована их контейнеризация с помощью Docker. Это позволило упаковать каждый сервис в изолированный контейнер. Это минимизирует зависимости от внешнего окружения и упрощает запуск на

любом сервере с установленным Docker, так как все необходимые настройки и установка библиотек производятся непосредственно внутри контейнера в изолированной среде. Для удобства запуска были добавлены конфигурации сервисов в Docker-compose.yml. Он позволяет прописать в нем все необходимые сервисы и упростить их запуск. Бэкенд основан на образе golang:1.24. Рабочая директория — ${GOPATH}/api/, далее копируется исходный код, выполняется сборка исполняемого файла с очисткой кэша. Контейнер занимает порт 8100 и запускает бинарный файл напрямую. Это обеспечивает компактность образа без лишних зависимостей. Фронтенд использует node:alpine, легковесный образ Node.js, в качестве базового образа. Рабочая директория — /app, копируются package.json и package-lock.json для установки зависимостей, далее копируется весь код. Занимается порт 5173 и далее производится запуск в dev-режиме. Для хранения секретов используется env файл, который указывается при конфигурации сервисов. Такой подход позволяет безопасно хранить такие важные данные, как пароли, API ключи, параметры запуска. Таким образом развертывание всех сервисов упрощается до выполнения команды «docker-compose up -d» в корне проекта.

Проект был успешно запущен и протестирован на сервере. Были выполнены ручные end-to-end сценарии. Была произведена загрузка в файла в формате .txt, анализ и обновление графа. Работу каждого этапа можно было отслеживать с помощью прогресс бара и промежуточных результатов. В результате был получен обновленный граф с новыми терминами, выделенными зеленым цветом.

Заключение

В рамках производственной практики, проведенной на базе ПАО «Сбербанк России», была успешно разработана система для анализа профессиональной литературы по кибербезопасности с целью выделения терминов и построения на их основе графа знаний. Разработанное решение

состоит из двух основных компонентов: фронтенд на React и серверной части на Go, интегрированных с использованием современных технологий и подходов, соответствующих новым стандартам. Фронтенд обеспечивает интуитивно понятный интерфейс для загрузки текстовых файлов, выбор модели для анализа, BERT или LightGBM, отображение промежуточных и финальных результатов обработки, а также визуализации графа знаний с использованием библиотеки vis-network. Дизайн интерфейса соответствует брендбуку Сбербанка. Выбор React оправдан его масштабируемостью, поддержкой большого сообщества и возможностями для создания высокопроизводительных приложений с обновлением в реальном времени через WebSocket. Серверная часть, реализованная на Go, выступает оркестратором, агрегирующим данные из внешних API. Он отвечает за выполнение всех стадий обработки файла: анализ текста, проверка терминов и обновление графа. Язык Go был выбран за высокую производительность, простоту развертывания и поддержку параллельных запросов, что является идеальным решением для обработки больших объемов данных в реальном времени. Архитектура сервера, основанная на принципах чистой архитектуры, обеспечивает модульность, тестируемость и масштабируемость. Использование Chi роутера, WebSocket и Swagger-документации делает сервер надежным и документируемым. Контейнеризация с использованием Docker и оркестрация через docker-compose упростили развертывание системы. Фронтенд и бэкенд упакованы в легковесные контейнеры с образами node:alpine и golang:1.24 соответственно, а docker-compose.yml обеспечивает автоматизированный запуск всех сервисов. Это позволило без проблем развернуть систему на удаленном сервере и обеспечить стабильную работу в production-режиме. Тестирование подтвердило корректность полного цикла работы системы от загрузки .txt файла до визуализации обновленного графа. Созданное решение готово к использованию для выявления новых терминов кибербезопасности и их визуального представления в виде графа знаний.